Rastro — Relatório parcial da AV1

GAT108 Automação Avançada · Projeto Integrador · Capacidades 1 a 5

《data da entrega》

1. Identificação

Grupo	Rastro (trio)
Integrantes	Juliano Cesar Ferreira Ramos · Andre Bueno Ocanha · João Paulo Silva
Papel nesta fase	《um parágrafo por integrante: o que cada um fez nas semanas 1 a 5》
Repositório	https://github.com/AlphaFNTZ/rastro
Commit entregue	《hash do commit enviado no e-mail》
Domínio	Comunicação descentralizada em áreas sem cobertura (rede mesh entre Androids)
Grandeza física da camada 1	Posição e velocidade GNSS com a acurácia informada pelo receptor; aceleração em 3 eixos

2. O que foi implementado

Levantado no histórico do Git (todos os ramos) em 21/10/2026.

João Paulo Silva — semana 1, commit `240077b` (13/09): `Posicao`, `IMURead`, `Trajeto` e `DomainExceptions` (exceções seladas `InvalidSensorReadException` e `InvalidTrajetoException`), com `PosicaoTest`, `IMUReadTest` e `TrajetoTest`; `7039bfe` (13/09) ajustes na proposta; `550d623` (15/09) registro de decisões da semana 1.

Juliano Cesar Ferreira Ramos — semana 2, `1613b5b`/`dc7ff92` (22/09): `CovarianciaGnss`, `IncertezaImu`, `CircularImuBuffer`, `ImuSensorSource`, `LocationMapper` e o teste de corrida de dez rodadas; semana 3, `09cb7ad`/`6713ba7` (24/09): `NearbyTransport`, `MeshRouter`, `RastroService` em primeiro plano; melhorias 01 a 03 e telas (`51ef36d` a `3ee9e20`, 24 a 30/09).

Andre Bueno Ocanha — 《preencher. Até 21/10/2026 não há commit com o endereço dele em nenhum ramo; se ele contribuiu por outro meio, descrever aqui e no e-mail》.

Implementação das semanas 4 e 5 (esta entrega) — pacote `com.example.rastro.av1`: `Matriz`, `Estatistica`, `RegistroFix`/`RegistroFixCsv`, `RegistroAquisicao`/`RegistroAquisicaoCsv`, `ProjecaoLocal`, `ModeloIncerteza`, `Reconciliador`, `DetectorErroGrosseiro`, `Janelas`, `InjetorFalhas`, `AnaliseEstatica`, `AnaliseAquisicao`, `TabelaResiduos`, `DemonstracaoDetector`, `EnsaioSensibilidade`, `SaidaEvidencias`, `ReprodutorRegistro`, `GravadorAquisicao` e `EnsaioActivity`; ligações em `RastroService`, `MainActivity`, `ConfiguracaoActivity`, `LocalizacaoSos` e no manifesto; 9 classes de teste em `app/src/test/.../av1`; scripts `av1_evidencias.ps1` e `av1_copiar_registros.ps1`. Código gerado com assistente de IA (seção 8). Revisado e comitado por: 《nome(s) e hash(es)》.

3. Fundamentação

3.1 Estruturas de capacidade fixa

`Trajeto` guarda no máximo `capacidade` posições e descarta a mais antiga: a memória não cresce com o tempo de operação, que num nó de borda é ilimitado. Comprimento é O(1); velocidade média e recorte temporal são uma varredura, O(n) com n ≤ `capacidade`. A inserção copia a janela (O(n)) porque `Trajeto` é

imutável — leitores concorrentes nunca veem um estado intermediário. Essa é a operação que poderia ser melhor: com um anel mutável protegido, como o `CircularImuBuffer`, a inserção cai para O(1).

`CircularImuBuffer` é o anel mutável: inserção O(1) sobre um vetor de tamanho fixo (500 leituras ≈ 10 s a 50 Hz), descarte contado do mais antigo quando lota, e `snapshot` / `drenar` em O(n) que devolvem uma **cópia**. Sem a cópia, o consumidor iteraria a mesma lista que o produtor está sobrescrevendo, e o erro apareceria como leitura fora de ordem ou `ConcurrentModificationException` intermitente — exatamente o tipo de falha que o teste de corrida procura.

3.2 Incerteza a partir do sensor

O Android informa a acurácia horizontal como um raio r68 que contém o fixo verdadeiro com 68% de probabilidade, e não como desvio padrão. Com erro gaussiano isotrópico de desvio σ por eixo, a distância ao centro segue uma Rayleigh, e $P(|e| \le r68) = 1 - \exp(-r68^2/(2\sigma^2)) = 0{,}68$, o que dá

$$\sigma^2 = r68^2 / (-2 \cdot \ln 0{,}32) \approx r68^2 / 2{,}279$$

onde σ^2 é a variância por eixo (m^2) e r68 o raio informado (m). A velocidade vem como módulo s (m/s) com acurácia σs e rumo θ (graus a partir do norte) com acurácia σθ. As componentes $vE = s \cdot \operatorname{sen}\theta$ e $vN = s \cdot \cos\theta$ têm covariância por propagação de primeira ordem $\Sigma v = J \cdot \mathrm{diag}(\sigma s^2, \sigma\theta^2) \cdot J^T$, em que $J = \partial(vE, vN)/\partial(s, \theta)$ é o jacobiano e σθ está em radianos. Parado, o Android não informa rumo; aí a direção é desconhecida e cada componente recebe média 0 e variância $\sigma s^2 + s^2/2$ (direção uniforme). Nenhum valor de incerteza é arbitrado: se o fixo não traz acurácia, ele não entra na reconciliação.

A dispersão medida com o aparelho parado (4.5) mostra se o receptor é honesto. A **razão de otimismo** é σ medido / σ declarado: acima de 1, o sensor promete mais precisão do que entrega e a matriz Σ fica pequena demais — a reconciliação passa a confiar demais nas posições e o detector dispara falsos alarmes (Apêndice A, linhas de ruído). O fator `fatorCovariancia` do `ModeloIncerteza` permite inflar Σ pela razão medida.

3.3 Reconciliação: a restrição física e por que ela é linear

Entre dois fixos consecutivos k e k+1, o deslocamento tem que ser compatível com a velocidade medida. Por eixo $a \in \{\text{leste, norte}\}$:

$$p_a(k+1) - p_a(k) - \tfrac{1}{2} \cdot (v_a(k) + v_a(k+1)) \cdot \Delta t_k = 0$$

onde $p_a(k)$ é a posição medida do fixo k naquele eixo (m, projeção local em torno do primeiro fixo da janela), $v_a(k)$ a componente da velocidade (m/s) e Δt_k o intervalo entre os fixos em **segundos** pelo relógio monotônico (`elapsedRealtimeNanos`). É a integração trapezoidal da velocidade. Como Δt é conhecido com precisão de nanossegundos, a equação é **linear nas medidas**. Empilhando as 4N medidas da janela no vetor y e as 2(N−1) restrições na matriz A, a restrição é $A \cdot y = 0$.

A reconciliação por mínimos quadrados ponderados resolve

$$\min (\hat{y} - y)^T \cdot \Sigma^{-1} \cdot (\hat{y} - y) \text{ sujeito a } A \cdot \hat{y} = 0 \Rightarrow \hat{y} = y - \Sigma \cdot A^T \cdot V^{-1} \cdot r$$

com ŷ as medidas reconciliadas, Σ a covariância das medidas (3.2), $r = A \cdot y$ o resíduo das restrições antes da reconciliação (m) e $V = A \cdot \Sigma \cdot A^T$ a covariância desse resíduo. A correção de cada medida, $y - \hat{y} = \Sigma \cdot A^T \cdot V^{-1} \cdot r$, é proporcional à variância dela: **a medida mais incerta é a que mais se move**. O teste `a medida mais incerta recebe a maior correcao` verifica isso com dois fixos de r68 = 2 m e 20 m: o segundo absorve cerca de 99% da correção. Depois da reconciliação $A \cdot \hat{y} = 0$ a menos de erro numérico (≈ 10^{-15} m), e a estatística $\gamma = r^T \cdot V^{-1} \cdot r$ segue χ^2 com m = 2(N−1) graus de liberdade se não houver erro grosseiro — com dados sintéticos calibrados, a média de γ/m em 300 janelas foi 0,98 (Apêndice A).

Alternativas descartadas. Ponderar todas as medidas igualmente (mínimos quadrados ordinários) distorceria justamente a medida mais confiável, puxando um fixo de 2 m tanto quanto um de 20 m. O filtro de Kalman estima um estado e exige um modelo de processo com ruído Q arbitrado; aqui o objetivo é

corrigir as medidas usando só o que o sensor informa. A restrição de distância $|\Delta p| = s \cdot \Delta t$ usaria o módulo da velocidade, mas é não linear e descarta a direção, que é o que detecta um salto lateral.

3.4 Erro grosseiro: critério quantitativo de corte

Com o ajuste $a = y - \hat{y}$ e sua covariância $W = \Sigma \cdot A^T \cdot V^{-1} \cdot A \cdot \Sigma$, cada medida j tem resíduo normalizado $z_j = |a_j| / \sqrt{W_{jj}}$, que segue N(0,1) sem erro grosseiro (teste de medida). Como uma janela de N = 10 fixos tem 40 medidas, testar cada uma a 1,96 daria $1 - 0{,}95^{40} \approx 87\%$ de falso alarme por janela. O limiar usa a correção de Šidák: $\beta = 1 - (1 - \alpha)^{(1/n)}$ e $z_{crítico} = \Phi^{-1}(1 - \beta/2)$, com $\alpha = 5\%$ a taxa de falso alarme por janela, n = 40 o número de medidas e Φ^{-1} o quantil da normal; resulta $z_{crítico} \approx 3{,}2$. Se max z_j passa do limiar, o fixo que contém a medida j é retirado e a janela é reconciliada de novo (eliminação serial, até 3). Sem o detector, a reconciliação não sabe que uma medida está errada e **espalha** o salto pelos vizinhos (4.7). Limitação: nos fixos das pontas há uma única restrição por eixo e todas as medidas dela têm o mesmo z; o erro é detectado, mas não localizado. Por isso o ensaio injeta saltos no interior e a reprodução usa janela deslizante, em que todo fixo passa pelo interior.

3.5 Concorrência e região crítica

O produtor é a `HandlerThread rastro-imu-50hz`, onde o `SensorManager` entrega as amostras; o consumidor é a thread `rastro-consumidor-imu` de um `ScheduledExecutorService` que drena a cada 1 s. A região crítica é o `CircularImuBuffer`: índices de início, quantidade, slots e o contador de perdas mudam juntos e precisam ser vistos juntos — uma operação composta. Usamos um único monitor (`synchronized`) porque todas as operações são curtas e nenhuma bloqueia dentro dele. Descartamos `ConcurrentLinkedQueue` (não limita a capacidade nem conta descarte de forma atômica com a inserção) e `ReentrantLock` (não precisamos de tentativa com tempo nem de várias condições). O `GravadorAquisicao` tem uma segunda região crítica, também com monitor único, entre a thread principal (GNSS), o consumidor (lote IMU) e a tarefa de 1 Hz.

O teste de corrida (10 rodadas, produtor, consumidor e observador simultâneos, 10 000 leituras num buffer de 64) verifica ordem, ausência de duplicata e consumidas + descartadas = produzidas. Ele **prova** que, nessas execuções, nenhuma intercalação produziu estado inconsistente; ele **não prova** ausência de corrida — só a análise do código (todo acesso dentro do monitor) sustenta isso. Rodadas sem falha são evidência empírica, não demonstração.

3.6 Aquisição periódica

A aquisição roda no `RastroService`, serviço em primeiro plano com tipo `connectedDevice|location` e notificação persistente: o Android não o encerra ao bloquear a tela. O GNSS pede um fixo a cada 1 s; o acelerômetro, 20 ms. O serviço em primeiro plano não impede o processador de dormir, e o acelerômetro é sensor não-wakeup: por isso o gravador mantém um `PARTIAL_WAKE_LOCK` durante a gravação. O log de 1 Hz registra, a cada segundo, leituras e taxa efetiva da IMU, maior intervalo entre amostras, fixos acumulados, tela ligada/bloqueio e bateria — é a evidência de que a tarefa periódica continuou. A política de escalonamento é a do Linux/Android (CFS, sem prioridade de tempo real), então o período é solicitado, não garantido; a medição de C_i e a análise de escalonabilidade são da AV2 (capacidade 6).

3.7 Ensaio reprodutível

O `InjetorFalhas` cria um `java.util.Random` por (semente, rótulo do ensaio). O algoritmo desse gerador é especificado na documentação Java, então a mesma semente produz os mesmos números em qualquer JVM ou Android, e a ordem em que os ensaios rodam não muda o resultado. Três degradações, as do Anexo A: **ruído** gaussiano extra nas posições sem mudar a acurácia declarada (simula um sensor otimista), **perda** de fixos (a janela encurta e Δt cresce) e **salto** de posição de magnitude conhecida (o erro grosseiro). A semente declarada é 20261020. O mesmo ensaio, rodado duas vezes, é comparado dentro do `EvidenciasAv1Test`; se diferir, o teste falha.

4. Evidências

Comandos: `scripts\av1_evidencias.ps1` (gera 4.2 a 4.8) e o roteiro de `docs/ENSAIO_AV1.md`.

4.1 Telas implementadas

⟪Inserir capturas: tela principal com a linha "Fixos GNSS · IMU · tempo"; notificação na tela de bloqueio; Configurações → Diagnóstico; tela Ensaio AV1 durante a reprodução com "ERRO GROSSEIRO". Legenda em cada figura.⟫

4.2 Saída do comando de teste

⟪Pendente: gerar `resultados/testes_unidade.md` com `scripts\av1_evidencias.ps1` depois da coleta (docs/ENSAIO_AV1.md)⟫

4.3 Teste de corrida — dez execuções

⟪Pendente: gerar `resultados/corrida_10_execucoes.md` com `scripts\av1_evidencias.ps1` depois da coleta (docs/ENSAIO_AV1.md)⟫

4.4 Aquisição com o aparelho bloqueado

⟪Pendente: gerar `resultados/aquisicao.md` com `scripts\av1_evidencias.ps1` depois da coleta (docs/ENSAIO_AV1.md)⟫

4.5 Dispersão das leituras estáticas e razão de otimismo

⟪Pendente: gerar `resultados/estatico.md` com `scripts\av1_evidencias.ps1` depois da coleta (docs/ENSAIO_AV1.md)⟫

⟪Pendente: figura `resultados/dispersao_estatica.svg` (gerada por EvidenciasAv1Test com o registro estático)⟫

4.6 Resíduo antes e depois da reconciliação

⟪Pendente: gerar `resultados/residuos.md` com `scripts\av1_evidencias.ps1` depois da coleta (docs/ENSAIO_AV1.md)⟫

4.7 Sem o detector de erro grosseiro, a reconciliação piora

⟪Pendente: gerar `resultados/demonstracao_detector.md` com `scripts\av1_evidencias.ps1` depois da coleta (docs/ENSAIO_AV1.md)⟫

4.8 Sensibilidade da detecção à degradação injetada

⟪Pendente: gerar `resultados/sensibilidade.md` com `scripts\av1_evidencias.ps1` depois da coleta (docs/ENSAIO_AV1.md)⟫

5. Integração

O projeto é independente: não consumimos código de outras duplas. Não há fakes em uso na aplicação. Nos testes, `MeshRouter` e o repasse de mensagens rodam sobre transportes falsos em memória (`ForwardNetwork`), e as classes da AV1 rodam na JVM com dados sintéticos (`Sintetico`) e com os registros reais. A ficha de proposta previa reconciliação com GNSS e detecção de falha injetada no GNSS isolando a medição (capacidade 2) e perda de dados com semente (capacidade 5): é o que foi entregue. A detecção de quedas saiu do escopo (D-21).

6. Autoria por integrante

《Repetir aqui, por integrante, as classes e métodos escritos e os hashes dos commits — a seção 2 já traz o levantamento do histórico até 21/10. Incluir os commits desta entrega.》

7. Limitações

- **Sem verdade de campo.** A referência da demonstração 4.7 é a reconciliação da janela limpa, não a posição verdadeira.
- **Erros de GNSS são correlacionados no tempo.** Fixos a 1 s não são independentes; γ não segue exatamente χ^2, e a dispersão estática de 30 min subestima erros de longo período.
- **Pontas da janela.** Um salto no primeiro ou no último fixo é detectado, mas não localizado (3.4).
- **IMU sem reconciliação.** A API padrão não informa σ por eixo do acelerômetro (D-07); a IMU entra só na aquisição periódica.
- **Período não garantido.** Android não é tempo real; C_i e escalonabilidade ficam para a AV2.
- **Energia.** Wake lock, GNSS a 1 s e IMU contínua aumentam o consumo, medido em 4.4.
- **Dado pessoal em claro.** O registro fica sem cifra no armazenamento do app até a AV2 (`AesGcmVault`, D-03); os registros publicados foram gravados no campus.
- **Processo.** 《Assumir com honestidade: a `main` ficou na semana 1 e não houve PR revisado nas semanas 2 a 5; não há integração contínua configurada.》
- **Compilação Android.** As partes Android desta entrega não foram compiladas no ambiente do assistente (sem SDK); foram compiladas e testadas por 《quem》 em 《data》 com `gradlew`.

8. Declaração de uso de IA

Usamos o assistente Claude (Anthropic) em 21/10/2026 para gerar o pacote `av1` (seção 2), as ligações no serviço e nas telas, os testes da AV1, os scripts, este rascunho de relatório e os slides. O assistente compilou e executou na JVM, no ambiente dele (Kotlin 2.0.21, JUnit 4.13.2), os testes de domínio, S2 e AV1 — 82 testes, zero falhas — e gerou o Apêndice A com dados sintéticos. Ele não teve acesso a aparelho nem aos dados físicos.

O que cada integrante verificou por conta própria: 《preencher por integrante — por exemplo: leu e explicou a fórmula de reconciliação; conferiu o sinal da matriz A; rodou `gradlew test`; fez a coleta de 30 min; conferiu os números do relatório contra os arquivos gerados》. 《Declarar também o uso de IA nas semanas 1 a 3, se houve.》

9. Leitura em camadas

Exigida somente na AV2.

Apêndice A — Verificação da implementação com dados sintéticos

> Não é evidência física. Caminhada simulada (`Sintetico.caminhada`, teste de unidade) com r68 = 4 m, 1,4 m/s, σ da velocidade 0,2 m/s, σ do rumo 5°, fixos a 1 s, e ruído coerente com a acurácia declarada. Serve para mostrar que a implementação se comporta como a teoria prevê antes de usar os dados reais. Semente 20261020; 20 janelas de 10 fixos × 20 repetições por linha.

Consistência estatística: média de γ/m em 300 janelas limpas = 0.983 (esperado 1, pois γ ~ $\chi^2(m)$).

Sensibilidade

Degradação	Parâmetro	Salto (m)	Ensaios	Inviáveis	Detecção	Localização correta	Falso alarme
salto	0.00	0	400	0	—	—	0%
salto	2.00	2	400	0	0%	0%	0%
salto	5.00	5	400	0	5%	5%	4%
salto	10.00	10	400	0	54%	51%	5%
salto	15.00	15	400	0	96%	96%	1%
salto	20.00	20	400	0	100%	100%	1%
salto	30.00	30	400	0	100%	100%	0%
salto	50.00	50	400	0	100%	100%	0%
salto	100.00	100	400	0	100%	100%	0%
ruído σ (m)	0.00	20	400	0	100%	100%	1%
ruído σ (m)	1.00	20	400	0	100%	99%	4%
ruído σ (m)	2.00	20	400	0	100%	98%	21%
ruído σ (m)	5.00	20	400	0	90%	78%	95%
ruído σ (m)	10.00	20	400	0	57%	32%	100%
perda (prob.)	0.00	20	400	0	100%	100%	1%
perda (prob.)	0.10	20	400	13	100%	100%	1%
perda (prob.)	0.20	20	400	32	100%	99%	1%
perda (prob.)	0.30	20	400	53	100%	100%	1%
perda (prob.)	0.50	20	400	161	100%	100%	0%

Menor salto com detecção ≥ 90%: 15.0 m.

Resíduo antes e depois (3 janelas)

Janela	Início (BRT)	Fixos	Duração (s)	RMS r antes (m)	máx |r| antes (m)	RMS r depois (m)	γ	χ^2 95%	Ajuste médio posição (m)	Eliminados pelo detector
1	—	10	9.0	2.764	4.826	2.2e-15	11.1	28.9	2.61	nenhum
2	—	10	9.0	3.438	6.514	6.3e-15	19.8	28.9	3.39	nenhum
3	—	10	9.0	4.433	8.730	3.6e-15	24.3	28.9	3.91	nenhum

Sem o detector a reconciliação piora (salto de 30 m)

Janela	Fixo com salto	Salto (m)	RMS vizinhos SEM detector (m)	RMS vizinhos COM detector (m)	Erro no fixo sem detector (m)	Detectou	Localizou
1	5	30	2.99	0.44	3.07	sim	sim
2	5	30	2.99	0.35	3.05	sim	sim
3	5	30	2.99	0.52	3.05	sim	sim
4	5	30	2.99	0.53	3.08	sim	sim
5	5	30	2.99	0.34	3.10	sim	sim

Ensaio estático: sensor calibrado × sensor otimista (ruído 1,6 × o declarado)

	Calibrado	Otimista
σ declarado por eixo (m)	3.31	3.31
σ medido (leste / norte, m)	3.32 / 3.24	5.27 / 5.26
Razão de otimismo	0.99	1.59
Fração dentro do próprio r68	69%	37%